RAG Debugging — Worked Example

> Back to the exercise · Round 1: Basic · Round 2: Intermediate · Round 3: Real-world · Round 4: Multi-agent · Solution

One scenario, followed across all 4 rounds: Doc08's retriever module (`chunking.py` + `retrieve()`). Each round is a different way "the right chunk never actually reaches the answer," at increasing distance from the obvious spot. Work through the rounds in order. Each ends with **"What do you think is wrong?"** — stop and actually answer before reading on.

- Round 1: Basic
- Round 2: Intermediate
- Round 3: Real-world
- Round 4: Multi-agent

Round: Basic

Setup: `chunk_by_chars()` , the simplest chunking method in `chunking.py` :

```
def chunk_by_chars(text, chunk_size):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end - 1])
        start = end
    return chunks
```

Symptoms: every chunk is missing its last character. Consistent, every run, every document.

Error output: no exception — it runs cleanly. A print of the chunks from `chunk_by_chars("The quick brown fox jumps", chunk_size=10)` :

```
chunk 0: 'The quick'
chunk 1: 'brown fox'
chunk 2: 'jumps'
```

Each chunk is only 9 characters long, and joining them gives `'The quickbrown foxjumps'` — the space at every chunk boundary is gone.

Expected vs. actual:

- Expected: chunks are exactly `chunk_size` characters each (except possibly the last one), and concatenating all of them reproduces the original text exactly.
- Actual: each chunk is one character short, and one character of the original text is silently dropped at every chunk boundary.

What do you think is wrong?

Round: Intermediate

Setup: chunking is fixed. The retriever module embeds documents at index time with one embedding model, and a later config change updates the *query-time* embedding model without updating the indexing side to match:

```
# indexing.py (run once, when documents were first loaded)
INDEX_EMBED_MODEL = "text-embedding-3-small"

# retrieve.py (used on every user question)
QUERY_EMBED_MODEL = "text-embedding-3-large"
```

Symptoms: looks, at first, like "the model just doesn't understand the documents" — a generation-layer problem. No exception is raised anywhere; every question returns *some* result, just consistently bad ones.

Log output:

```
[retrieve] question: "What is our refund window?"
[retrieve] top-3 results (scores): [0.11, 0.09, 0.08]
[retrieve] returned chunks appear unrelated to the question
```

Scores that used to reliably sit around 0.7-0.9 for genuinely relevant chunks are now uniformly low, for every question, against every document.

Expected vs. actual:

- Expected: a question about the refund policy retrieves the chunk that actually discusses refunds, with a high similarity score.
- Actual: retrieval quality degraded across the board, for every question, right after a routine-looking config change to which embedding model queries use.

What do you think is wrong?

Round: Real-world

Setup: indexing and querying both use the same embedding model again. Documents are chunked with a fixed character count, same as Round 1 but now fixed and working correctly.

Symptoms: only sometimes happens — most questions against the knowledge base work fine. One specific, real question keeps coming back with a half-right answer, and only that one.

Log output:

```
[retrieve] question: "How many days do customers have to request a refund?"
[retrieve] top-1 chunk:
  "...our return process is designed to be simple. Customers"
[retrieve] top-2 chunk:
  "have 30 days from the date of purchase to request a full refund..."
[generation] answer: "I don't see a specific number of days mentioned
  in the provided context."
```

Expected vs. actual:

- Expected: the retriever surfaces the chunk (or chunks) containing the actual fact, and the model answers "30 days" confidently.
- Actual: the one sentence containing the fact ("Customers have 30 days...") got cut in half by a chunk boundary — the retriever does return both halves in its top results, but the generation step, looking at each retrieved chunk somewhat independently, never connects "Customers" (end of chunk 1) to "have 30 days..." (start of chunk 2) as one continuous sentence.

What do you think is wrong?

Round: Multi-agent

Setup: chunking is now genuinely correct, retrieval genuinely returns the right chunks with good scores. In Project 4, the Research agent calls this retriever as one of its tools, gets back the right chunks (with scores and source metadata), but only writes a plain-text summary of what it found into shared state — the scores and source info aren't part of `research_findings` at all.

Symptoms: testing the retriever alone, directly, returns exactly the right chunk every time, with a high confidence score. Running the full pipeline, the Analysis agent (which is supposed to only use facts it can trace back to a source) occasionally flags a correct fact as "unverified" and asks the Writer to soften it or drop it — making the final answer *less* confident than the retrieved evidence actually supports.

Log output:

```
[research_agent] retrieve() returned: [
  {"text": "Customers have 30 days to request a refund.",
     "score": 0.91, "source": "policy.md"}
]
[research_agent] research_findings written to shared state:
  "Customers can request a refund within 30 days."
[analysis_agent] checking research_findings for traceable sources...
[analysis_agent] no source metadata found — flagging as UNVERIFIED
[writer_agent] softened claim per analysis flag:
  "refunds may be available within a certain window"
```

Expected vs. actual:

- Expected: a fact the retriever found with high confidence and a named source flows through the pipeline as verified, and the final answer states it plainly.
- Actual: the retriever did its job correctly, but the source and score never made it past the Research agent's own node — Analysis, working only from the plain-text summary in shared state, has no way to tell a well-sourced fact from an unsourced guess, and downgrades it — invisible testing the retriever or the Research agent's tool call alone.

What do you think is wrong?

Full solution: Show me the solution